

Real-Time Financial Fraud Detection System

Solution Architecture Report

Prepared for: Cloud Native Architecture Project
Scenario: Major financial institution fraud detection uplift
Submission format: PDF

Name: Yu Zhang

Requirement	Design Response
Real-time fraud analysis	Event-driven ingestion, streaming enrichment, low-latency model serving, and synchronous decision response for authorisation paths.
Peak 1,000 TPS	Kafka partitioning, stateless horizontally scaled microservices, backpressure, and autoscaling from CPU plus queue lag.
Replayable transactions	Immutable transaction and decision event log, archived to object storage with retention and schema versioning.
Elastic scaling	Kubernetes HPA/KEDA, managed Kafka partition scaling, separate online and analytic workloads.
Fraud analyst frontend	BFF, case management API, WebSocket/SSE push alerts, cached dashboards.
Operational observability	OpenTelemetry traces/log correlation, Prometheus metrics, SLO dashboards and alerting.

1. Executive Summary

The bank is replacing an aging batch-driven fraud detection platform with a cloud-native, event-driven system that can score transactions in near real time, preserve all relevant events for replay and audit, and provide immediate alerts for fraud analysts. The proposed design separates transaction ingestion, validation, enrichment, scoring, decisioning, alerting, case management, and reporting into independently scalable microservices.

The core architectural choice is to make Kafka or an equivalent managed streaming platform the durable backbone of the system. The ingestion path records each transaction event before downstream processing, so the bank can replay, investigate, and satisfy compliance obligations. Low-latency services process events asynchronously where possible, while the authorisation decision uses a controlled synchronous response path because payment authorisation requires a deterministic answer within a tight time budget.

2. Problem Understanding and Technical Objectives

Business problem	Technical objective	Architectural implication
Batch fraud detection is too slow	Classify transactions within seconds and return hold/approve/decline decisions quickly	Online scoring service, feature cache, bounded service latency, real-time alerting
Regulatory requirement for replayability	Capture every transaction, model score, decision, and alert with immutable history	Append-only event log, object storage archive, schema registry, audit metadata
Peak volume of 1,000 TPS	Scale without permanent peak cost	Partitioned streams, stateless services, autoscaling, performance SLOs
Analysts need live visibility	Provide responsive frontend and high-confidence alert notification	BFF, push notifications, case store, paginated queries, dashboard cache

Financial services risk profile	Strong security, resilience, observability, governance	mTLS, RBAC, encryption, idempotency, disaster recovery, SLO alerts
---------------------------------	--	--

3. Solution Architecture

The solution uses event-driven microservices for high-throughput processing while keeping a synchronous decision response for payment authorisation. This hybrid approach avoids over-coupling all services while still meeting the business need to approve, decline, or hold a transaction quickly.

Solution Architecture - Real-Time Fraud Detection

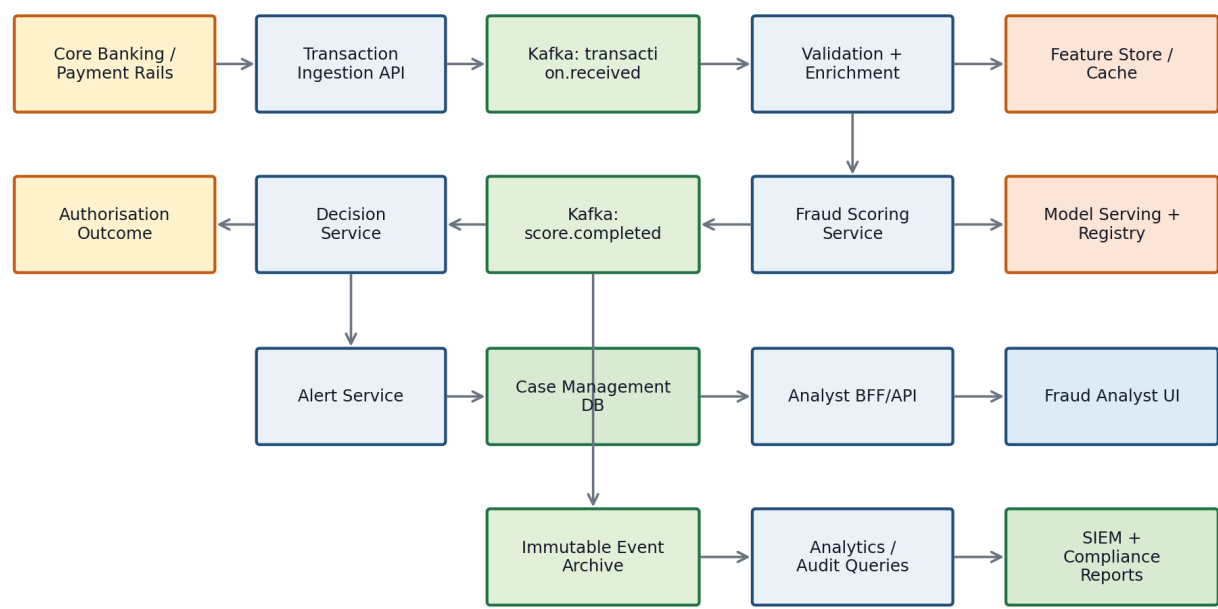


Figure 1. Component-level solution architecture.

4. Functional Decomposition into Microservices

Each microservice has a single primary responsibility and communicates through versioned contracts. The design intentionally favours stateless services for online processing, because stateless services are easier to scale horizontally and recover after failures.

Microservice	Primary responsibility	Inputs	Outputs / triggers	Coupling notes
Transaction Ingestion API	Authenticate upstream channels, validate envelope, assign correlation IDs, persist the first event	POST /transactions or payment rail message	TransactionReceived event; synchronous ACK	Coupled to payment rail contract only; downstream is decoupled through Kafka
Validation Service	Validate required fields, risk policy eligibility, idempotency and duplicate detection	TransactionReceived	TransactionValidated or TransactionRejected	Consumes events; no direct dependency on scoring
Enrichment Service	Add customer, merchant, device, geo, velocity and account context	TransactionValidated; feature cache; customer/profile APIs	TransactionEnriched	Uses feature store and cache; fallback strategy for stale features

Fraud Scoring Service	Call model serving endpoint and generate risk score and reason codes	TransactionEnriched; model endpoint; feature vector	FraudScoreComputed	Latency-sensitive; model version is explicit in event
Decision Service	Apply policy thresholds, compliance rules and model result to approve, decline, or hold	FraudScoreComputed; policy rules	FraudDecisionMade; response to payment rail	Requires deterministic policy versioning and idempotent decisions
Alert Service	Create analyst alerts for high-confidence fraud or uncertain high-value cases	FraudDecisionMade with HOLD/DECLINE or high risk score	AlertRaised; CaseCreated	Asynchronous; failure should not block payment outcome
Case Management Service	Maintain case lifecycle, analyst actions, comments	AlertRaised; analyst commands	CaseUpdated; AnalystActionRecorded	Stateful service backed by relational/document database

	and evidence			
Analyst BFF	Optimise frontend API, aggregate case, alert and metrics views	Frontend requests; WebSocket subscriptions	Read models, alert streams, dashboard summaries	Frontend-specific adapter prevents UI coupling to internal services
Audit and Replay Service	Archive events, replay transaction history, support compliance evidence	All domain events	Replay jobs; audit reports	Read-only replay path isolated from online scoring
Model Governance Service	Register model versions, approval status, drift signals and rollback metadata	Model package, evaluation reports, drift metrics	ModelPublished; ModelRolledBack	Separates ML lifecycle from transaction services

5. Event-Driven Communication Design

Event Flow and Service Triggers

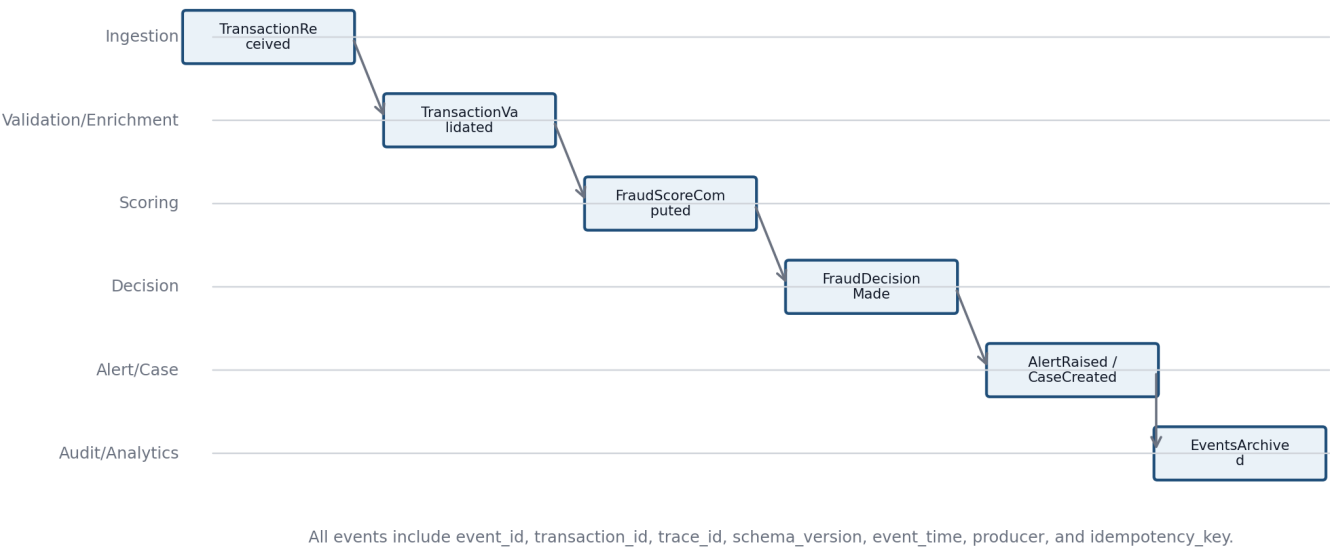


Figure 2. Domain events, service triggers, and event progression.

The primary event broker is Kafka or a managed Kafka-compatible service. Topics are partitioned by transaction_id or account_id depending on ordering needs. For fraud detection, account-level ordering is preferred for velocity features, while transaction-level partitioning maximises parallelism. The recommended compromise is to partition online scoring by account_id and use sufficient partitions to support 1,000 TPS with headroom.

Event	Published by	Consumed by	When / why	Minimal schema fields
TransactionReceived	Transaction Ingestion API	Validation, Audit	A payment transaction arrives and is durably accepted	event_id, transaction_id, account_id, amount, currency, merchant_id, channel, event_time, trace_id

TransactionValidated	Validation Service	Enrichment , Audit	Envelope and business validation pass	validation_result, idempotency_key, risk_eligible, validation_version
TransactionEnriched	Enrichment Service	Fraud Scoring, Audit	Context and feature values are attached	feature_vector_ref, feature_snapshot_time, customer_segment, merchant_risk_band
FraudScoreComputed	Fraud Scoring Service	Decision, Analytics, Audit	ML model returns a score and explanation	score, confidence, reason_codes, model_version, inference_latency_ms
FraudDecisionMade	Decision Service	Alert, Case, Payment Outcome, Audit	Policy converts score into APPROVE, DECLINE or HOLD	decision, policy_version, threshold, decision_latency_ms
AlertRaised	Alert Service	BFF, Case Management, SIEM	High-confidence fraud or review case needs analyst action	alert_id, severity, transaction_id, case_id, SLA_due_at

CaseUpdated	Case Management	BFF, Audit, Analytics	Analyst changes case status or disposition	case_id, status, analyst_id_hash, action, timestamp
-------------	-----------------	-----------------------	--	---

5.1 Event Schema Guidance

All events use a common envelope: event_id, event_type, schema_version, producer, event_time, trace_id, span_id, idempotency_key, source_system, and payload. Payload fields are schema-registered and backward compatible. Personally identifiable information should be tokenised or minimised wherever possible, with sensitive attributes encrypted and access-controlled.

Deviation from pure event-driven design: the bank should keep a synchronous interaction between the payment rail and the Decision Service for the final authorisation outcome. This is justified because payment processing must return an answer within a bounded time. All important steps are still evented for replay and audit.

6. End-to-End Transaction Flow

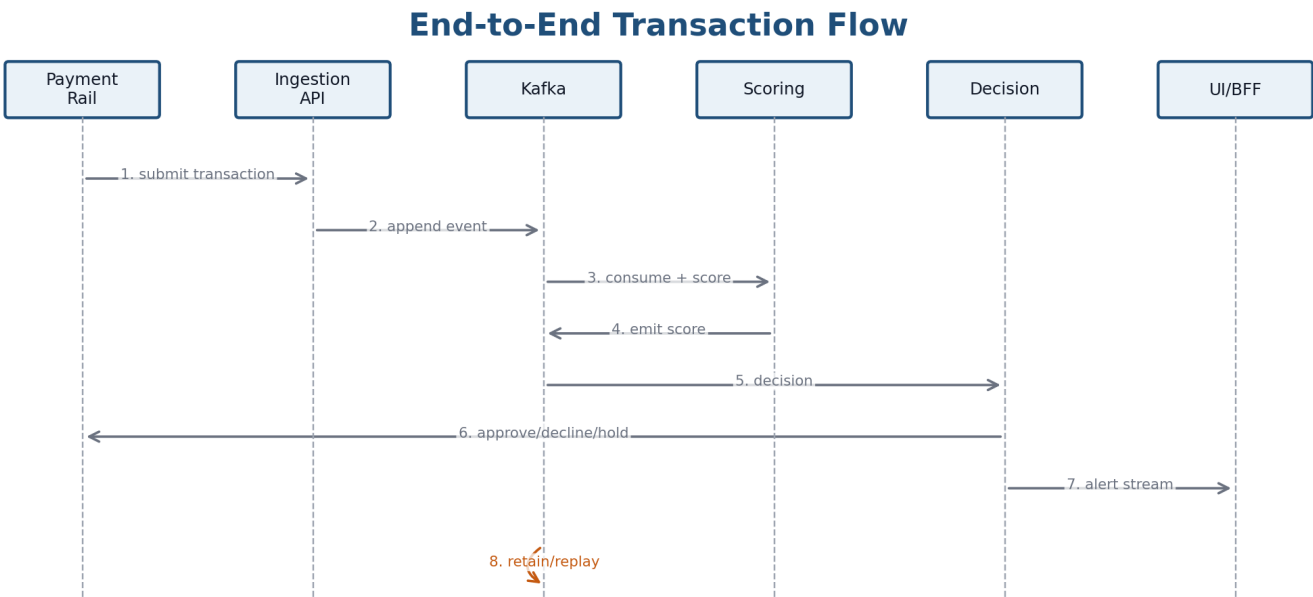


Figure 3. End-to-end transaction journey from payment rail to analyst alert.

Step	Description	Reliability control
1	Payment rail submits transaction to the ingestion API.	mTLS, OAuth2/client credentials, rate limiting, idempotency key
2	Ingestion API validates envelope, assigns trace_id, writes TransactionReceived.	Durable append before ACK; retry-safe producer
3	Validation and enrichment add context and feature references.	Dead-letter topic for malformed events; feature fallback
4	Scoring service calls model serving and emits score with model version.	Timeout budget, circuit breaker, champion/challenger model routing
5	Decision service applies thresholds and rules.	Deterministic policy version, idempotent decision store
6	Payment outcome is returned: approve, decline or hold.	Synchronous service path with strict latency SLO
7	High-risk results produce alert and case creation.	Async retry, outbox pattern, analyst notification
8	All events are retained and archived for replay.	Retention policy, object storage archive, audit checksum

7. Frontend and BFF Architecture

The analyst frontend must remain responsive during normal operations and during audit-heavy periods. It should not query every microservice directly. A

Backend-for-Frontend (BFF) provides aggregated, frontend-friendly APIs and manages subscriptions for real-time alerts.

Concern	Design choice	Rationale
Live alerts	WebSocket or Server-Sent Events from BFF	Pushes high-confidence fraud events immediately without constant polling
Dashboard performance	Materialised read model and cached aggregates	Avoids expensive joins over operational services during peak investigations
Case search	Paginated query API backed by indexed case store and audit read model	Supports analyst workflow and compliance lookup
Security	Role-based access, step-up authentication for sensitive actions, full audit trail	Aligns with financial services controls
Resilience	UI degrades gracefully: alerts remain visible even if analytics widgets fail	Separates critical alerting from nice-to-have dashboard panels

8. Observability Design

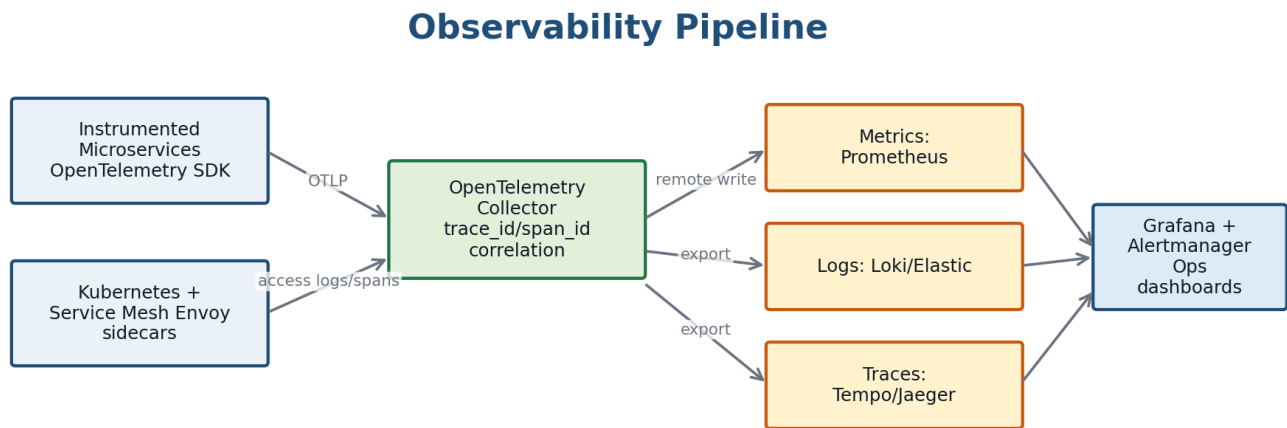


Figure 4. Logs, metrics and traces are correlated through shared trace_id and span_id.

All services emit OpenTelemetry traces, structured JSON logs and dimensional metrics. The trace_id is injected at ingestion and propagated through Kafka headers, HTTP/gRPC calls, model requests and database interactions. Logs include trace_id, span_id, transaction_id hash, event_id, service_name, environment, model_version and policy_version. Metrics are scraped or exported via OpenTelemetry Collector to Prometheus-compatible storage, while traces and logs are exported to dedicated backends.

Telemetry type	Collection method	Important data points	Correlation
Metrics	OpenTelemetry SDK and Prometheus scraping	TPS, event lag, inference latency, decision latency, error rate, DLQ count, CPU/memory, cache hit ratio	service_name, route/topic, result, model_version

Logs	Structured JSON to collector, then Loki/Elastic/SIEM	validation failures, model timeout, policy decision, analyst actions, security events	trace_id, span_id, event_id, transaction_id_hash
Traces	OpenTelemetry auto/manual instrumentation	End-to-end transaction span, Kafka consume/produce spans, DB spans, model inference spans	W3C trace context propagated in headers and Kafka metadata

8.1 Key Metrics and Proposed SLOs

SLI / Metric	Proposed SLO	Alert threshold	Why it matters
End-to-end decision latency	99% of eligible transaction decisions completed in < 800 ms over 5 minutes	p99 > 800 ms for 5 min or p95 > 500 ms for 10 min	Controls customer/payment experience and real-time compliance
Scoring service inference latency	99% of model inference calls completed in < 200 ms over 5 minutes	p99 > 200 ms or timeout rate > 0.5%	Model serving is the most latency-sensitive dependency
Event processing freshness	99.9% of TransactionReceived events consumed by scoring within < 2 seconds	consumer lag equivalent > 2 seconds for 5 min	Indicates whether streaming path is keeping up

Decision error rate	99.95% of decision requests complete without 5xx or unhandled exception per 30 days	5xx rate > 0.05% for 5 min	Financial decision service must be highly reliable
Alert delivery latency	99% of AlertRaised events visible in analyst UI in < 3 seconds	p99 > 3 seconds for 10 min	Supports real-time analyst intervention
Replay integrity	100% of archived events pass checksum and schema validation daily	Any failed checksum or missing sequence	Required for audit and compliance evidence

9. Proposed Technology Stack

Layer	Recommended technologies	Justification / trade-off
Compute and orchestration	Managed Kubernetes: AKS/EKS/GKE/OpenShift; containers; HPA/KEDA	Portable cloud-native platform, pod-level scaling; operational complexity mitigated by managed service
Service mesh	Istio or Linkerd with Envoy sidecars	mTLS, traffic policy, retries, telemetry; adds operational overhead and must be tuned to avoid latency inflation
Streaming	Managed Kafka, Confluent Cloud, MSK, Event Hubs Kafka API, or Redpanda	Durable replayable log, partition scaling, consumer groups; requires schema governance and partition planning

API layer	Spring Boot/Java or .NET for APIs; gRPC for internal low-latency calls	Mature enterprise ecosystem, observability support, strong typing
ML serving	KServe/Seldon/BentoML or cloud ML endpoints; ONNX/Triton for optimised inference	Independent model deployment and autoscaling; GPU use only if justified by model complexity
Feature store/cache	Feast or managed feature store; Redis for hot features	Consistent online/offline features and low-latency lookup; requires freshness monitoring
Operational data	PostgreSQL for cases; Redis for cache; object storage for immutable archive; analytical warehouse/lakehouse for reporting	Separates transactional, cache, archive and analytic workloads
Observability	OpenTelemetry, Prometheus, Grafana, Loki/Elastic, Tempo/Jaeger, Alertmanager, SIEM integration	Vendor-neutral instrumentation and rich correlation across logs, metrics, traces
Security	Vault/KMS, network policies, OAuth2/OIDC, mTLS, secrets rotation, encryption at rest and in transit	Financial-grade controls; adds governance processes but reduces breach and compliance risk
CI/CD and governance	GitHub Actions/Azure DevOps/GitLab, Terraform, Helm, Argo CD, policy-as-code, model approval gates	Repeatable deployments, audit trail and controlled release of code and models

10. Architectural Patterns and Their Contribution

Pattern	How it is used	Benefits	Costs / risks
Microservices	Separate ingestion, enrichment, scoring, decision, alerting, case and audit services	Independent scaling, team ownership, fault isolation	Distributed complexity, contract testing, eventual consistency
Event-driven architecture	Kafka topics carry durable domain events between services	Replayability, decoupling, elasticity, backpressure handling	Requires schema governance, idempotent consumers, careful ordering
BFF	Analyst BFF aggregates read models and manages alert streams	Responsive frontend, lower chatty API traffic, UI-specific security	Additional service to operate and version
Caching	Redis for hot feature lookups and dashboard aggregates	Low latency, reduced backend load	Stale data risk; must define TTL and cache invalidation
Service mesh	mTLS, retries, timeouts, telemetry and policy enforcement	Security and observability without duplicating code	Extra latency and operational learning curve
Outbox / inbox idempotency	State changes and event publication are made retry-safe	Prevents duplicate cases and supports recovery	More implementation effort and storage overhead

CQRS/read models	Operational writes separated from analyst search and dashboards	Keeps UI fast and reduces production DB load	Read models are eventually consistent
------------------	---	--	---------------------------------------

11. Scalability and Resilience Design

Deployment Topology and Scaling

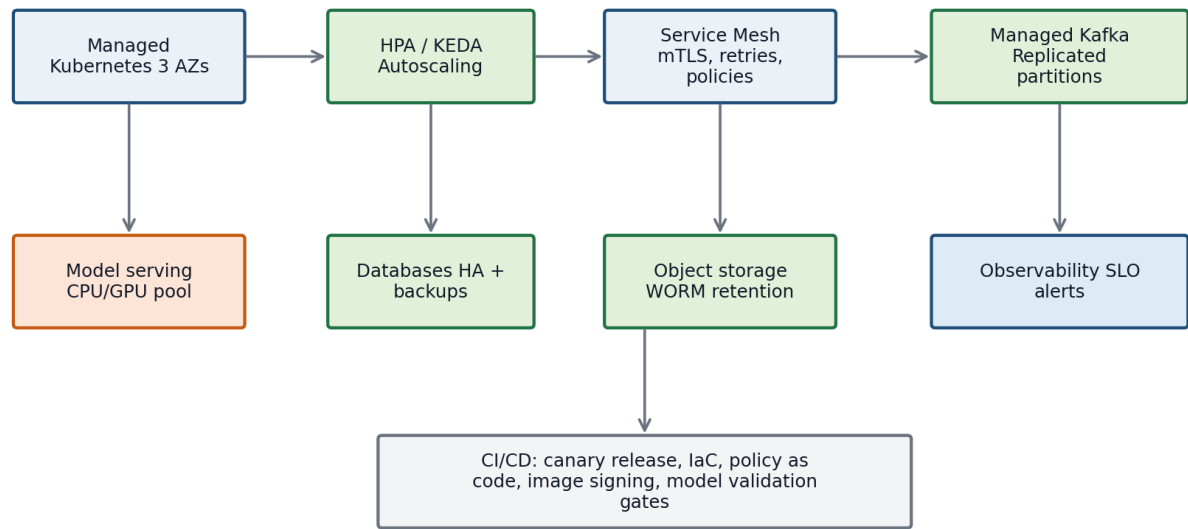


Figure 5. Deployment topology with autoscaling, high availability, and operational controls.

11.1 Scaling Strategy

Component	Scaling approach	Notes
Individual microservices	Horizontal pod autoscaling based on CPU, memory, request rate and custom latency metrics	Stateless services can scale independently; decision service has higher minimum replicas

Fraud scoring	Scale model serving replicas using inference latency, queue depth and GPU/CPU utilisation	Use model warm pools to avoid cold starts during spikes
Event messaging	Increase topic partitions, consumer group replicas and broker capacity	Partition key must preserve required ordering without creating hot partitions
Analytics workloads	Run separately from online processing using read replicas, lakehouse or warehouse jobs	Prevents audit/report queries from starving real-time scoring
Frontend/BFF	Scale BFF by concurrent sessions and request latency; cache dashboard read models	Keeps analyst UI responsive under alert spikes

11.2 Failure Modes and Mitigations

Failure mode / SPOF risk	Impact	Mitigation	Data loss / restore approach
Kafka broker or zone failure	Event processing delay or partition unavailability	Managed Kafka across AZs, replication factor 3, min in-sync replicas, producer acks=all	Committed records remain available if replicas survive; restore from object archive if disaster-level loss occurs
Model serving outage or latency spike	Transactions cannot be scored in time	Circuit breaker, fallback risk rules, cached safe defaults, hot standby champion model	Decision event records fallback reason; replay affected transactions

			through model later
Feature store/cache outage	Poor score quality or scoring latency	Redis HA, local fallback for low-risk rules, asynchronous refresh, feature freshness monitoring	Persist feature snapshot reference; replay with corrected features if required
Case database outage	Alerts cannot create cases	Outbox queue, retry with exponential backoff, read-only UI degradation	AlertRaised remains in Kafka; rebuild cases by replaying alerts
BFF/UI outage	Analysts cannot view alerts	Multi-replica BFF, CDN/static UI hosting, fallback SIEM notification	No transaction data loss; alerts remain in broker and case store
Observability backend outage	Reduced incident visibility	Buffer telemetry in collector, independent critical health checks	Business events unaffected; temporary telemetry gap accepted with alert on collector drop rate
Incorrect model deployment	False positives/negatives at scale	Model registry approval, canary, shadow evaluation, rollback, drift and outcome monitoring	Keep model_version in each decision; rerun post-incident analysis and remediation

12. Security, Privacy and Compliance Considerations

Financial fraud systems process sensitive payment and customer data, so security controls must be part of the architecture rather than added later. All traffic should use mTLS inside the cluster and TLS at boundaries. Sensitive attributes should be tokenised, encrypted, masked in logs, and protected by strict RBAC/ABAC. Analyst actions are audit events. Model outputs should be explainable enough to support investigations and avoid black-box operational decisions.

Control area	Design recommendation
Identity and access	OIDC integration, least privilege service accounts, analyst roles, privileged action approval
Data protection	Encryption in transit and at rest, key management through KMS/Vault, PII minimisation and tokenisation
Auditability	Immutable event archive, schema versioning, model/policy version logged for every decision
Network security	Private subnets, network policies, service mesh mTLS, ingress WAF and rate limits
Model governance	Approval workflows, champion/challenger testing, explainability fields, drift alerts, rollback
Data retention	Defined retention for event log, archive storage, case records and telemetry; WORM storage where required

13. Critical Evaluation of Trade-offs

Decision	Why it is worthwhile	Trade-off / challenge
Event-driven core	Durable, replayable and scalable foundation for compliance and volume spikes	Eventual consistency and more complex debugging than a monolith

Hybrid synchronous decision path	Payment authorisation needs a bounded answer, not only background processing	Synchronous path must be protected with timeouts and fallbacks
Service mesh	Consistent mTLS, retries, traffic policy and telemetry across services	Can add latency and operational overhead; policies require careful tuning
Caching hot features	Supports low-latency scoring at 1,000 TPS	Stale data can affect scores; freshness SLO is needed
Separate audit replay path	Compliance replay cannot disrupt live scoring	Duplicates infrastructure and requires data lineage discipline
Managed cloud services	Reduces operational burden and improves HA	Vendor-specific limits and cost model must be reviewed; exit strategy needed

14. Realistic Deployment Guidance

In production, the system should be deployed to a managed Kubernetes platform across at least three availability zones, with managed Kafka and managed databases where possible. On Azure, this could map to AKS, Azure Event Hubs with Kafka API or Confluent Cloud, Azure Database for PostgreSQL, Azure Cache for Redis, Azure Machine Learning managed endpoints, Azure Monitor/Grafana and ADLS with immutable storage policies. Equivalent mappings exist on AWS and GCP.

For audit periods, compliance reviews and end-of-financial-year spikes, the bank should pre-scale Kafka partitions and consumer limits, run synthetic 1,000 TPS plus headroom load tests, schedule analytics workloads away from online scoring, warm model-serving replicas, temporarily increase BFF and read-model capacity, and use feature flags to disable non-critical dashboard widgets if the system is under pressure.

15. Architecture Decision Summary

ADR	Decision	Status	Consequence
ADR-01	Use Kafka-compatible streaming as the durable system backbone	Accepted	Supports replayability, decoupling and high-volume processing
ADR-02	Use microservices aligned to ingestion, enrichment, scoring, decision, alerting and audit	Accepted	Enables independent scaling and clear ownership
ADR-03	Keep payment decision response synchronous but event-recorded	Accepted	Meets authorisation latency while retaining audit trail
ADR-04	Adopt OpenTelemetry across all services	Accepted	Provides consistent traces, logs and metrics correlation
ADR-05	Use BFF and CQRS read models for analyst UI	Accepted	Protects core services and keeps frontend responsive
ADR-06	Use service mesh for mTLS and traffic controls	Conditional	Adopt if operational maturity is sufficient; otherwise start with ingress and library-level controls

16. References

- Confluent Documentation, Apache Kafka Message Delivery Guarantees, including at-most-once, at-least-once and exactly-once semantics.
- OpenTelemetry Documentation, Observability Primer and logging specification describing traces, metrics, logs and trace-context correlation.
- Kubernetes Documentation, Horizontal Pod Autoscaling for workload scaling based on resource or custom metrics.

17.

How We Collect Logs
How We Collect Metrics
How We Collect Traces
Trace Visualization

17. Observability and Rich Telemetry

17.1 Observability Objectives

The fraud detection platform must provide complete visibility into transaction processing, service health, machine learning performance, and infrastructure behavior.

The observability solution is based on the three pillars of observability:

1. Logs
2. Metrics
3. Distributed Traces

Using these three telemetry signals together allows operations teams to detect incidents, investigate failures, and understand the complete journey of a transaction through the platform.

17.2 How We Collect Logs

Every microservice generates structured JSON logs.

Examples include:

- Transaction received
- Validation failures
- Fraud scores generated
- Fraud decisions made
- Alert generation
- Analyst actions
- System errors

Example:

```
{  
  "timestamp": "2026-01-22T10:30:15Z",  
  "service": "fraud-scoring-service",  
  "level": "INFO",  
  "trace_id": "xyz789",  
  "transaction_id": "tx12345",  
  "message": "Fraud scoring completed",  
  "fraud_score": 0.92  
}
```

Log Collection Flow

Microservice



Container stdout



Kubernetes Node



OpenTelemetry Collector / Fluent Bit



Centralized Logging Platform



Grafana / Kibana

All logs are shipped to a centralized logging platform such as Elasticsearch or Grafana Loki.

Centralized logging allows engineers to search for events across all services without needing direct access to individual containers.

17.3 How We Collect Metrics

Metrics provide numerical measurements about system performance.

Each microservice exposes a Prometheus-compatible `/metrics` endpoint.

Prometheus continuously scrapes these endpoints.

Metrics Collection Flow

Microservice

↓

`/metrics` endpoint

↓

Prometheus

↓

Time-Series Database

↓

Grafana Dashboards

Application Metrics

Examples include:

- Transactions processed per second
- Fraud score requests per second
- Average fraud scoring latency
- Decision latency
- Error rate

Infrastructure Metrics

Collected from Kubernetes and nodes:

- CPU utilization
- Memory utilization
- Pod restarts
- Network throughput

Kafka Metrics

Collected from brokers and consumers:

- Consumer lag
- Topic throughput
- Failed message processing

17.4 How We Collect Distributed Traces

Distributed tracing follows a transaction through multiple services.

When a transaction first enters the platform, OpenTelemetry generates a unique Trace ID.

Example:

Trace ID = abc123

The Trace ID is propagated through:

- HTTP Headers
- gRPC Metadata
- Kafka Message Headers

Each service creates a Span.

Example:

Trace: abc123

Span 1 -> Transaction Ingestion

Span 2 -> Validation Service

Span 3 -> Enrichment Service

Span 4 -> Fraud Scoring Service

Span 5 -> Decision Service

Span 6 -> Alert Service

Trace Collection Flow

Microservices

↓

OpenTelemetry SDK

↓

OpenTelemetry Collector

↓

Jaeger / Grafana Tempo

↓

Trace Visualization

This allows engineers to identify exactly where latency or failures occur.

17.5 Correlating Logs, Metrics and Traces

A key design requirement is the ability to correlate all telemetry signals.

The Trace ID is used as the common identifier.

Logs

```
{  
  "trace_id": "abc123",  
  "transaction_id": "tx567"  
}
```

Traces

Trace = abc123

Metrics

```
fraud_scoring_latency_ms  
  
service="fraud-scoring"
```

model_version="v2.1"

Example Investigation

An alert fires:

Fraud Scoring Latency > 200ms

The engineer performs the following steps:

Metric Alert

↓

Open Dashboard

↓

View Related Trace

↓

Find Slow Span

↓

Search Logs Using Trace ID

↓

Identify Root Cause

For example:

1. Prometheus detects increased fraud scoring latency.
2. Grafana displays the alert.
3. The engineer opens the trace associated with the slow request.
4. The trace shows the Fraud Scoring Service consuming 85% of total latency.
5. Using the same Trace ID, the engineer retrieves logs from the service.
6. Logs show the model server was waiting for feature store responses.
7. Root cause is identified without manually inspecting multiple services.

This correlation significantly reduces Mean Time To Resolution (MTTR) and supports rapid incident response.

17.6 Key Metrics and Performance Characteristics

The following metrics are critical for monitoring system performance:

Metric	Purpose
End-to-end transaction latency	Measure customer experience
Fraud scoring latency	Monitor ML inference performance
Transaction throughput (TPS)	Verify support for 1,000 TPS
Kafka consumer lag	Verify real-time processing
Fraud detection rate	Measure model effectiveness
Error rate	Measure platform reliability

SLOs

SLO	Target
End-to-end decision latency	99% < 800ms
Fraud scoring latency	99% < 200ms
Alert visibility in UI	99% < 3s
Event processing delay	99.9% < 2s
Decision API availability	99.95%